SVKM's NMIMS

Mukesh Patel School of Technology, Management and Engineering

Department of Computer Engineering

CSEDS, Semester V

Coursera Course: Database and SQL for Data Science with Python

Case Study Assignment (30 Marks)

Part 1: Cover page

Field	**Details**
Student name	Aadit Rangnekar
Roll number	48
Odd / Even	Even
Case study allotted	B - QuickBite Online Food Delivery
Database used	MySQL
Date of submission	21 October 2026

Part 2: Schema design

The QuickBite scenario is represented by eight related MySQL tables. IDs and quantities use INT, money uses DECIMAL, dates use DATE or DATETIME, and text uses VARCHAR or TEXT. The types and constraints are listed below.

1. Customers

Column	Data type	Constraints
customer_id	INT	PRIMARY KEY
customer_name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
phone	VARCHAR(20)	NOT NULL
city	VARCHAR(100)	NOT NULL
join_date	DATE	NOT NULL

2. Restaurants

Column	Data type	Constraints
restaurant_id	INT	PRIMARY KEY
restaurant_name	VARCHAR(100)	NOT NULL
city	VARCHAR(100)	NOT NULL
cuisine_type	VARCHAR(100)	NOT NULL
avg_rating	DECIMAL(3,2)	NOT NULL, CHECK 0 to 5

3. MenuItems

Column	Data type	Constraints
item_id	INT	PRIMARY KEY
restaurant_id	INT	NOT NULL, FK to Restaurants.restaurant_id
item_name	VARCHAR(100)	NOT NULL
category	VARCHAR(20)	NOT NULL, CHECK Veg / Non-Veg
price	DECIMAL(10,2)	NOT NULL, CHECK > 0

4. DeliveryAgents

Column	Data type	Constraints
agent_id	INT	PRIMARY KEY
agent_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	NOT NULL
vehicle_type	VARCHAR(100)	NOT NULL
city	VARCHAR(100)	NOT NULL

5. Orders

Column	Data type	Constraints
order_id	INT	PRIMARY KEY
customer_id	INT	NOT NULL, FK to Customers.customer_id
restaurant_id	INT	NOT NULL, FK to Restaurants.restaurant_id
agent_id	INT	NOT NULL, FK to DeliveryAgents.agent_id
order_date	DATETIME	NOT NULL
order_status	VARCHAR(20)	NOT NULL, CHECK Placed / Delivered / Cancelled

6. OrderItems

Column	Data type	Constraints
order_item_id	INT	PRIMARY KEY
order_id	INT	NOT NULL, FK to Orders.order_id
item_id	INT	NOT NULL, FK to MenuItems.item_id
quantity	INT	NOT NULL, CHECK > 0

Additional constraint: UNIQUE (order_id, item_id).

7. Payments

Column	Data type	Constraints
payment_id	INT	PRIMARY KEY
order_id	INT	NOT NULL, UNIQUE, FK to Orders.order_id
payment_mode	VARCHAR(20)	NOT NULL, CHECK UPI / Card / Cash / Wallet
amount	DECIMAL(10,2)	NOT NULL, CHECK >= 0
payment_status	VARCHAR(20)	NOT NULL, CHECK Success / Failed

8. Reviews

Column	Data type	Constraints
review_id	INT	PRIMARY KEY
order_id	INT	NOT NULL, UNIQUE, FK to Orders.order_id
customer_id	INT	NOT NULL, FK to Customers.customer_id
rating	INT	NOT NULL, CHECK 1 to 5
comments	TEXT	Optional

Relationships

Customers, restaurants and delivery agents link to many orders; restaurants offer many menu items. OrderItems links orders to items. Each order has one payment and an optional review; reviews also reference customers.

ER diagram

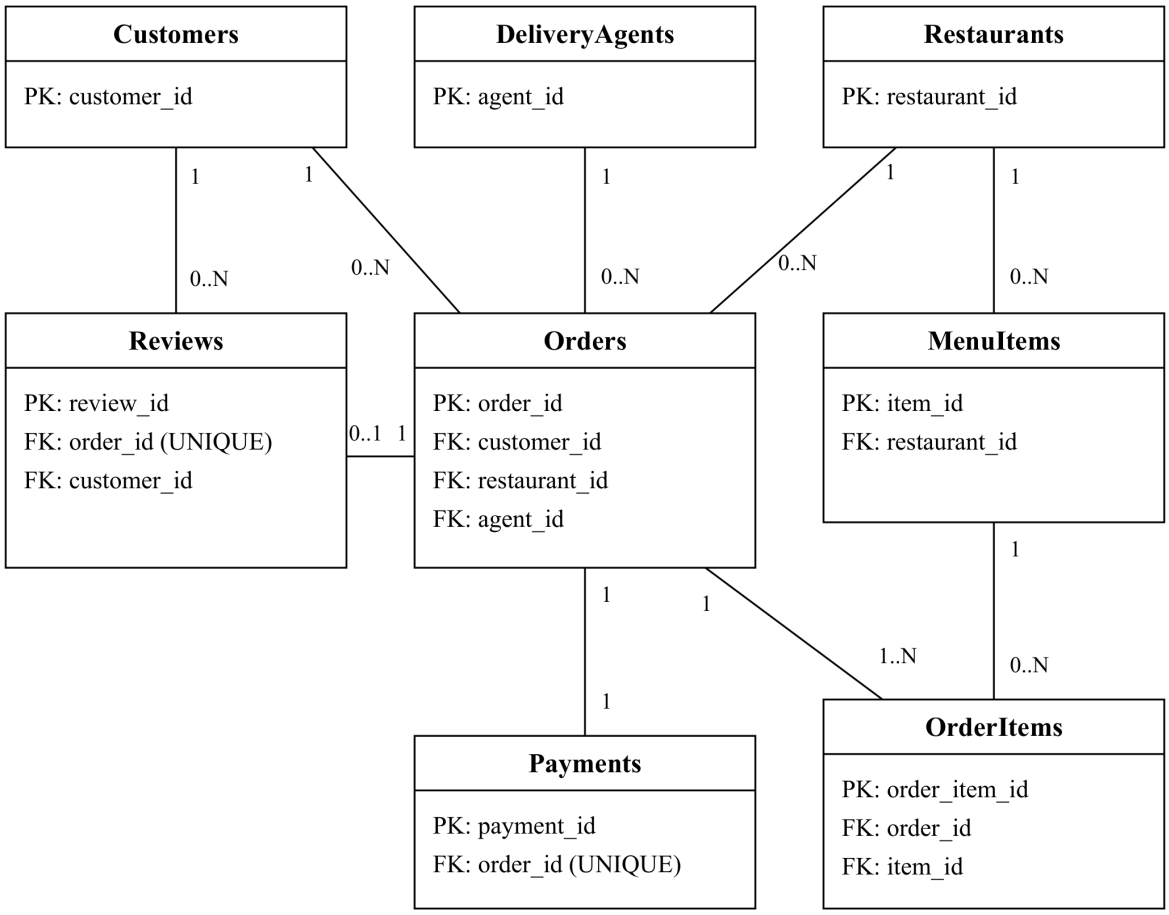


Figure 1: ER diagram (primary and foreign keys only).

1 = one; 0..N = zero or many; 1..N = one or many; 0..1 = optional one.

Payments.order_id and Reviews.order_id are UNIQUE. Every seeded order has a payment and at least one item; reviews are optional.

Part 3: Python-SQL connectivity code

I used mysql.connector to connect Python to the local MySQL server and pandas to display results. Connection settings are loaded from the private .env file, so the password is not embedded in the code or report.

Code:

```
import mysql.connector
import pandas as pd
import os
from dotenv import load_dotenv

load_dotenv(".env", override=True)
database = os.getenv("MYSQL_DATABASE", "quickbite_roll48")
conn = mysql.connector.connect(
    host=os.getenv("MYSQL_HOST", "127.0.0.1"),
    port=int(os.getenv("MYSQL_PORT", "3306")),
    user=os.getenv("MYSQL_USER", "root"),
    password=os.environ["MYSQL_PASSWORD"], connection_timeout=5
)
cursor = conn.cursor()
cursor.execute(f"CREATE DATABASE IF NOT EXISTS {database} CHARACTER SET
    utf8mb4")
cursor.execute(f"USE {database}")
print(f"Connected to MySQL {conn.server_info}; database: {database}")

def show(sql):
    cursor.execute(sql)
    return pd.DataFrame(cursor.fetchall(), columns=cursor.column_names)
```

Output (VS Code screenshot):

```
Connected to MySQL 9.6.0; database: quickbite_roll48
```

Reset assignment database

This guarded cell drops and recreates only quickbite_roll48. Running the notebook from the beginning therefore creates fresh data for each screenshot session.

Code:

```
if database != "quickbite_roll48":
    raise ValueError("Reset is allowed only for quickbite_roll48.")
cursor.execute("DROP DATABASE IF EXISTS quickbite_roll48")
cursor.execute("CREATE DATABASE quickbite_roll48 CHARACTER SET utf8mb4")
cursor.execute("USE quickbite_roll48")
print("Reset quickbite_roll48. Ready to create all 8 tables.")
```

Output (VS Code screenshot):

```
Reset quickbite_roll48. Ready to create all 8 tables.
```

Part 4: Table creation code

Parent tables are created before child tables. The dictionary contains the eight definitions; the loop executes them using InnoDB and SHOW TABLES lists the resulting tables.

Code:

```
ddl = {
    "Customers": """
        customer_id INT PRIMARY KEY,
        customer_name VARCHAR(100) NOT NULL,
        email VARCHAR(255) NOT NULL UNIQUE,
        phone VARCHAR(20) NOT NULL, city VARCHAR(100) NOT NULL,
        join_date DATE NOT NULL""",
    "Restaurants": """
        restaurant_id INT PRIMARY KEY,
        restaurant_name VARCHAR(100) NOT NULL,
        city VARCHAR(100) NOT NULL, cuisine_type VARCHAR(100) NOT NULL,
        avg_rating DECIMAL(3,2) NOT NULL CHECK (avg_rating BETWEEN 0 AND 5)""",
    "MenuItems": """
        item_id INT PRIMARY KEY, restaurant_id INT NOT NULL,
        item_name VARCHAR(100) NOT NULL,
        category VARCHAR(20) NOT NULL CHECK (category IN ('Veg', 'Non-Veg')),
        price DECIMAL(10,2) NOT NULL CHECK (price > 0),
        FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)""",
    "DeliveryAgents": """
        agent_id INT PRIMARY KEY, agent_name VARCHAR(100) NOT NULL,
        phone VARCHAR(20) NOT NULL, vehicle_type VARCHAR(100) NOT NULL,
        city VARCHAR(100) NOT NULL""",
```

Table creation code (continued)

```
    "Orders": """
        order_id INT PRIMARY KEY, customer_id INT NOT NULL,
        restaurant_id INT NOT NULL, agent_id INT NOT NULL,
        order_date DATETIME NOT NULL,
        order_status VARCHAR(20) NOT NULL
            CHECK (order_status IN ('Placed', 'Delivered', 'Cancelled')),
        FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
        FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id),
        FOREIGN KEY (agent_id) REFERENCES DeliveryAgents(agent_id)""",
    "OrderItems": """
        order_item_id INT PRIMARY KEY, order_id INT NOT NULL,
        item_id INT NOT NULL, quantity INT NOT NULL CHECK (quantity > 0),
        UNIQUE (order_id, item_id),
        FOREIGN KEY (order_id) REFERENCES Orders(order_id),
        FOREIGN KEY (item_id) REFERENCES MenuItems(item_id)""",
    "Payments": """
        payment_id INT PRIMARY KEY, order_id INT NOT NULL UNIQUE,
        payment_mode VARCHAR(20) NOT NULL
            CHECK (payment_mode IN ('UPI', 'Card', 'Cash', 'Wallet')),
        amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
        payment_status VARCHAR(20) NOT NULL
            CHECK (payment_status IN ('Success', 'Failed')),
        FOREIGN KEY (order_id) REFERENCES Orders(order_id)""",
    "Reviews": """
        review_id INT PRIMARY KEY, order_id INT NOT NULL UNIQUE,
        customer_id INT NOT NULL,
        rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5), comments TEXT,
        FOREIGN KEY (order_id) REFERENCES Orders(order_id),
        FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)"""
}
for table, definition in ddl.items():
    cursor.execute(f"CREATE TABLE IF NOT EXISTS {table} ({definition})
    ENGINE=InnoDB")
display(show("SHOW TABLES"))
```

Output (VS Code screenshot):

	Tables_in_quickbite_roll48
0	Customers
1	DeliveryAgents
2	MenuItems
3	OrderItems
4	Orders
5	Payments
6	Restaurants
7	Reviews

Part 5: Data insertion code

The 46 original fictional records span Mumbai, Pune and Nashik. Tables are populated in the prescribed order, with at least five rows each. Payment amounts match item totals; failed payments represent attempted amounts. Parameterized inserts are committed together or rolled back on error.

Code:

```
data = {
    "Customers": [
        (1, "Anaya Deshmukh", "anaya48@example.test", "0000004801", "Mumbai",
    "2026-09-01"),
        (2, "Vihaan Shah", "vihaan48@example.test", "0000004802", "Mumbai",
    "2026-09-03"),
        (3, "Ira Kulkarni", "ira48@example.test", "0000004803", "Pune",
    "2026-09-05"),
        (4, "Kabir Patil", "kabir48@example.test", "0000004804", "Nashik",
    "2026-09-07"),
        (5, "Sana Shaikh", "sana48@example.test", "0000004805", "Nashik",
    "2026-09-09")],
    "Restaurants": [
        (1, "Harbour Bowl", "Mumbai", "Indian", "4.40"),
        (2, "Ember Slice", "Mumbai", "Italian", "4.20"),
        (3, "Pune Wok", "Pune", "Chinese", "4.30"),
        (4, "Godavari Kitchen", "Nashik", "Maharashtrian", "4.50"),
        (5, "Leaf and Ladle", "Nashik", "South Indian", "4.10")],
    "MenuItems": [
        (1, 1, "Paneer Rice Bowl", "Veg", "200.00"),
        (2, 1, "Chicken Rice Bowl", "Non-Veg", "300.00"),
        (3, 2, "Margherita Pizza", "Veg", "350.00"),
        (4, 3, "Chilli Chicken", "Non-Veg", "280.00"),
        (5, 4, "Misal Pav", "Veg", "150.00"),
        (6, 5, "Masala Dosa", "Veg", "120.00")],
    "DeliveryAgents": [
        (1, "Omkar Jadhav", "0000004811", "Scooter", "Mumbai"),
        (2, "Rhea Naik", "0000004812", "Bike", "Mumbai"),
        (3, "Dev More", "0000004813", "Scooter", "Pune"),
        (4, "Meera Kale", "0000004814", "Bike", "Nashik"),
        (5, "Adil Khan", "0000004815", "Bicycle", "Nashik")],
```

Data insertion code (continued)

```
    "Orders": [
        (1, 1, 1, 1, "2026-10-10 12:15:00", "Delivered"),
        (2, 1, 1, 1, "2026-10-11 19:20:00", "Delivered"),
        (3, 2, 2, 2, "2026-10-12 13:00:00", "Delivered"),
        (4, 2, 2, 2, "2026-10-13 20:10:00", "Delivered"),
        (5, 3, 3, 3, "2026-10-14 18:30:00", "Delivered"),
        (6, 4, 4, 4, "2026-10-15 11:45:00", "Cancelled")],
    "OrderItems": [
        (1, 1, 1, 1), (2, 1, 2, 1), (3, 2, 1, 2), (4, 2, 2, 1),
        (5, 3, 3, 2), (6, 4, 3, 1), (7, 5, 4, 1), (8, 6, 5, 4)],
    "Payments": [
        (1, 1, "Cash", "500.00", "Success"),
        (2, 2, "UPI", "700.00", "Success"),
        (3, 3, "Card", "700.00", "Success"),
        (4, 4, "UPI", "350.00", "Success"),
        (5, 5, "Wallet", "280.00", "Success"),
        (6, 6, "Card", "600.00", "Failed")],
    "Reviews": [
        (1, 1, 1, 4, "Fresh bowl and timely delivery."),
        (2, 2, 1, 5, "Good portions for a shared meal."),
        (3, 3, 2, 4, "Pizza arrived hot."),
        (4, 4, 2, 3, "Good taste; packaging could improve."),
        (5, 5, 3, 5, "Spice level was just right.")]
}

try:
    for table, rows in data.items():
        marks = ", ".join(["%s"] * len(rows[0]))
        cursor.executemany(f"INSERT INTO {table} VALUES ({marks})", rows)
    conn.commit()
    print("Committed 46 rows across 8 tables.")
except Exception:
    conn.rollback()
    raise
```

Output (VS Code screenshot):

```
Committed 46 rows across 8 tables.
```

Contents of every table

Code:

```
for table in data:
    print(f"{table}: {len(data[table])} rows")
    display(show(f"SELECT * FROM {table} ORDER BY 1"))
```

Table: Customers

	customer_id	customer_name	email	phone	city	join_date
0	1	Anaya Deshmukh	anaya48@example.test	0000004801	Mumbai	2026-09-01
1	2	Vihaan Shah	vihaan48@example.test	0000004802	Mumbai	2026-09-03
2	3	Ira Kulkarni	ira48@example.test	0000004803	Pune	2026-09-05
3	4	Kabir Patil	kabir48@example.test	0000004804	Nashik	2026-09-07
4	5	Sana Shaikh	sana48@example.test	0000004805	Nashik	2026-09-09

Table: Restaurants

	restaurant_id	restaurant_name	city	cuisine_type	avg_rating
0	1	Harbour Bowl	Mumbai	Indian	4.40
1	2	Ember Slice	Mumbai	Italian	4.20
2	3	Pune Wok	Pune	Chinese	4.30
3	4	Godavari Kitchen	Nashik	Maharashtrian	4.50
4	5	Leaf and Ladle	Nashik	South Indian	4.10

Table: MenuItems

	item_id	restaurant_id	item_name	category	price
0	1	1	Paneer Rice Bowl	Veg	200.00
1	2	1	Chicken Rice Bowl	Non-Veg	300.00
2	3	2	Margherita Pizza	Veg	350.00
3	4	3	Chilli Chicken	Non-Veg	280.00
4	5	4	Misal Pav	Veg	150.00
5	6	5	Masala Dosa	Veg	120.00

Table: DeliveryAgents

	agent_id	agent_name	phone	vehicle_type	city
0	1	Omkar Jadhav	0000004811	Scooter	Mumbai
1	2	Rhea Naik	0000004812	Bike	Mumbai
2	3	Dev More	0000004813	Scooter	Pune
3	4	Meera Kale	0000004814	Bike	Nashik
4	5	Adil Khan	0000004815	Bicycle	Nashik

Table: Orders

	order_id	customer_id	restaurant_id	agent_id	order_date	order_status
0	1	1	1	1	2026-10-10 12:15:00	Delivered
1	2	1	1	1	2026-10-11 19:20:00	Delivered
2	3	2	2	2	2026-10-12 13:00:00	Delivered
3	4	2	2	2	2026-10-13 20:10:00	Delivered
4	5	3	3	3	2026-10-14 18:30:00	Delivered
5	6	4	4	4	2026-10-15 11:45:00	Cancelled

Table: OrderItems

	order_item_id	order_id	item_id	quantity
0	1	1	1	1
1	2	1	2	1
2	3	2	1	2
3	4	2	2	1
4	5	3	3	2
5	6	4	3	1
6	7	5	4	1
7	8	6	5	4

Table: Payments

	payment_id	order_id	payment_mode	amount	payment_status
0	1	1	Cash	500.00	Success
1	2	2	UPI	700.00	Success
2	3	3	Card	700.00	Success
3	4	4	UPI	350.00	Success
4	5	5	Wallet	280.00	Success
5	6	6	Card	600.00	Failed

Table: Reviews

	review_id	order_id	customer_id	rating	comments
0	1	1	1	4	Fresh bowl and timely delivery.
1	2	2	1	5	Good portions for a shared meal.
2	3	3	2	4	Pizza arrived hot.
3	4	4	2	3	Good taste; packaging could improve.
4	5	5	3	5	Spice level was just right.

Part 6: Query execution and results

Each query uses GROUP BY and at most one join. Q1, Q3 and Q4 use joins; Q2 and Q3 use HAVING. Together they use COUNT, AVG, SUM and MAX. Monetary results are in INR.

Query 1

(a) Question: Show each restaurant's name, the number of menu items it offers and the average price of those items.

(b) SQL query:

```
SELECT r.restaurant_name, COUNT(m.item_id) AS menu_items,
       ROUND(AVG(m.price), 2) AS average_price
FROM Restaurants r
LEFT JOIN MenuItems m ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY r.restaurant_id
```

(c) Python execution cell:

```
q1 = show("""
SELECT r.restaurant_name, COUNT(m.item_id) AS menu_items,
       ROUND(AVG(m.price), 2) AS average_price
FROM Restaurants r
LEFT JOIN MenuItems m ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY r.restaurant_id
""")
display(q1)
```

(d) Output (VS Code screenshot):

	restaurant_name	menu_items	average_price
0	Harbour Bowl	2	250.00
1	Ember Slice	1	350.00
2	Pune Wok	1	280.00
3	Godavari Kitchen	1	150.00
4	Leaf and Ladle	1	120.00

Rows returned: 5

LEFT JOIN preserves every restaurant. COUNT counts item IDs, and GROUP BY produces one row per restaurant.

Query 2

(a) Question: Show each payment mode and the total amount collected through it. Display only payment modes whose total amount is more than INR 500.

(b) SQL query:

```
SELECT payment_mode, SUM(amount) AS total_collected
FROM Payments
WHERE payment_status = 'Success'
GROUP BY payment_mode
HAVING SUM(amount) > 500
ORDER BY total_collected DESC, payment_mode
```

(c) Python execution cell:

```
q2 = show("""
SELECT payment_mode, SUM(amount) AS total_collected
FROM Payments
WHERE payment_status = 'Success'
GROUP BY payment_mode
HAVING SUM(amount) > 500
ORDER BY total_collected DESC, payment_mode
""")
display(q2)
```

(d) Output (VS Code screenshot):

	payment_mode	total_collected
0	UPI	1050.00
1	Card	700.00

Rows returned: 2

Collected means successful payments only. WHERE removes failed attempts before aggregation; HAVING filters grouped totals. Cash totals exactly INR 500 and is excluded.

Query 3

(a) Question: Show each customer's name and the number of orders they placed. Display only customers who placed 2 or more orders.

(b) SQL query:

```
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) >= 2
ORDER BY c.customer_id
```

(c) Python execution cell:

```
q3 = show("""
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) >= 2
ORDER BY c.customer_id
""")
display(q3)
```

(d) Output (VS Code screenshot):

	customer_name	order_count
0	Anaya Deshmukh	2
1	Vihaan Shah	2

Rows returned: 2

The join links customers to orders. HAVING includes the boundary of exactly two orders. All order statuses count as placed orders.

Query 4

(a) Question: Show each delivery agent's name and the number of orders assigned to them.

(b) SQL query:

```
SELECT a.agent_name, COUNT(o.order_id) AS assigned_orders
FROM DeliveryAgents a
LEFT JOIN Orders o ON a.agent_id = o.agent_id
GROUP BY a.agent_id, a.agent_name
ORDER BY a.agent_id
```

(c) Python execution cell:

```
q4 = show("""
SELECT a.agent_name, COUNT(o.order_id) AS assigned_orders
FROM DeliveryAgents a
LEFT JOIN Orders o ON a.agent_id = o.agent_id
GROUP BY a.agent_id, a.agent_name
ORDER BY a.agent_id
""")
display(q4)
```

(d) Output (VS Code screenshot):

	agent_name	assigned_orders
0	Omkar Jadhav	2
1	Rhea Naik	2
2	Dev More	1
3	Meera Kale	1
4	Adil Khan	0

Rows returned: 5

LEFT JOIN includes the unassigned agent. COUNT(o.order_id) returns zero for that agent. Cancelled orders still count as assignments.

Query 5

(a) Question: Show each menu category (Veg / Non-Veg), the number of items in it and the highest item price.

(b) SQL query:

```
SELECT category, COUNT(*) AS item_count, MAX(price) AS highest_price
FROM MenuItems
GROUP BY category
ORDER BY category
```

(c) Python execution cell:

```
q5 = show("""
SELECT category, COUNT(*) AS item_count, MAX(price) AS highest_price
FROM MenuItems
GROUP BY category
ORDER BY category
""")
display(q5)
```

(d) Output (VS Code screenshot):

	category	item_count	highest_price
0	Non-Veg	2	300.00
1	Veg	4	350.00

Rows returned: 2

GROUP BY separates the menu into categories. COUNT and MAX summarize each category without a join.

Closing the connection

Code:

```
cursor.close()
conn.close()
print("MySQL connection closed.")
```

Output (VS Code screenshot):

```
MySQL connection closed.
```

Summary of results

Query	Output	Rows returned	Interpretation
Q1	**Restaurant** / **Items** / **Average price** Harbour Bowl / 2 / 250.00 Ember Slice / 1 / 350.00 Pune Wok / 1 / 280.00 Godavari Kitchen / 1 / 150.00 Leaf and Ladle / 1 / 120.00	5	Harbour Bowl offers 2 items; the other restaurants offer 1 each.
Q2	**Payment mode** / **Total collected** UPI / 1050.00 Card / 700.00	2	UPI collected INR 1050 and Card INR 700; both exceed INR 500.
Q3	**Customer** / **Orders** Anaya Deshmukh / 2 Vihaan Shah / 2	2	Anaya Deshmukh and Vihaan Shah each placed 2 orders.
Q4	**Agent** / **Orders** Omkar Jadhav / 2 Rhea Naik / 2 Dev More / 1 Meera Kale / 1 Adil Khan / 0	5	Omkar and Rhea have 2 assignments each; Adil has none.
Q5	**Category** / **Items** / **Highest price** Non-Veg / 2 / 300.00 Veg / 4 / 350.00	2	Veg has 4 items (maximum INR 350); Non-Veg has 2 (maximum INR 300).

Part 7: Conclusion

I translated the QuickBite scenario into eight related MySQL tables.

Primary keys, foreign keys and checks helped maintain valid data.

Python connected to MySQL and executed creation, insertion and analytical SQL.

Joins, aggregates and GROUP BY summarized records; HAVING filtered groups.

I resolved the connection issue by running locally in VS Code with .env settings.

A guarded reset cell allowed fresh runs without duplicate-key errors.